

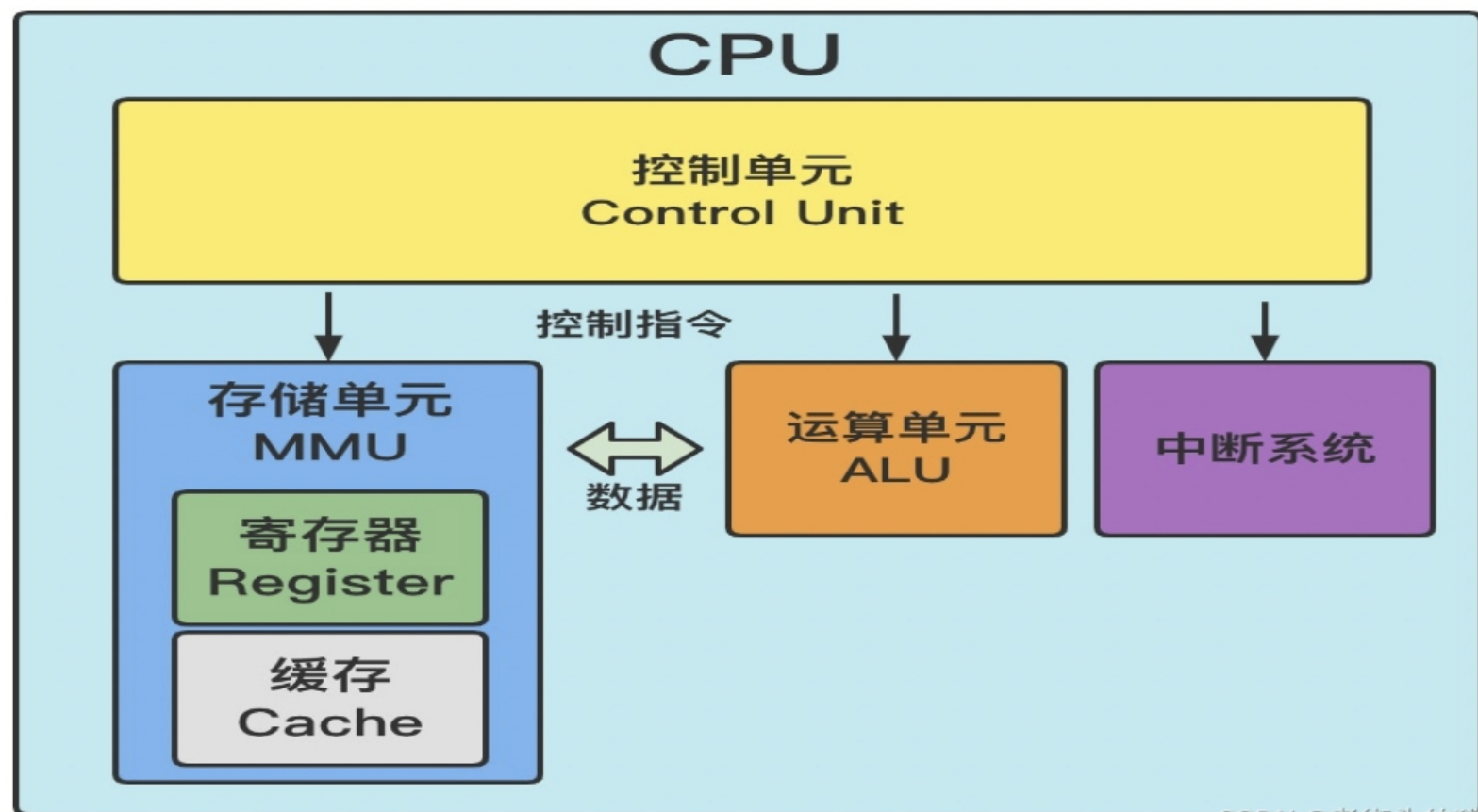
CSP-J——C++基础知识

基础知识与编程环境

CPU（Central Processing Unit）：中央处理器，是计算机的中枢神经系统。

内存（Memory）：随机存取存储器（RAM），它是一种用于临时存储数据和程序的计算机组件。

硬盘（Hard Disk Drive）：用于存储和检索数据的非易失性存储设备。

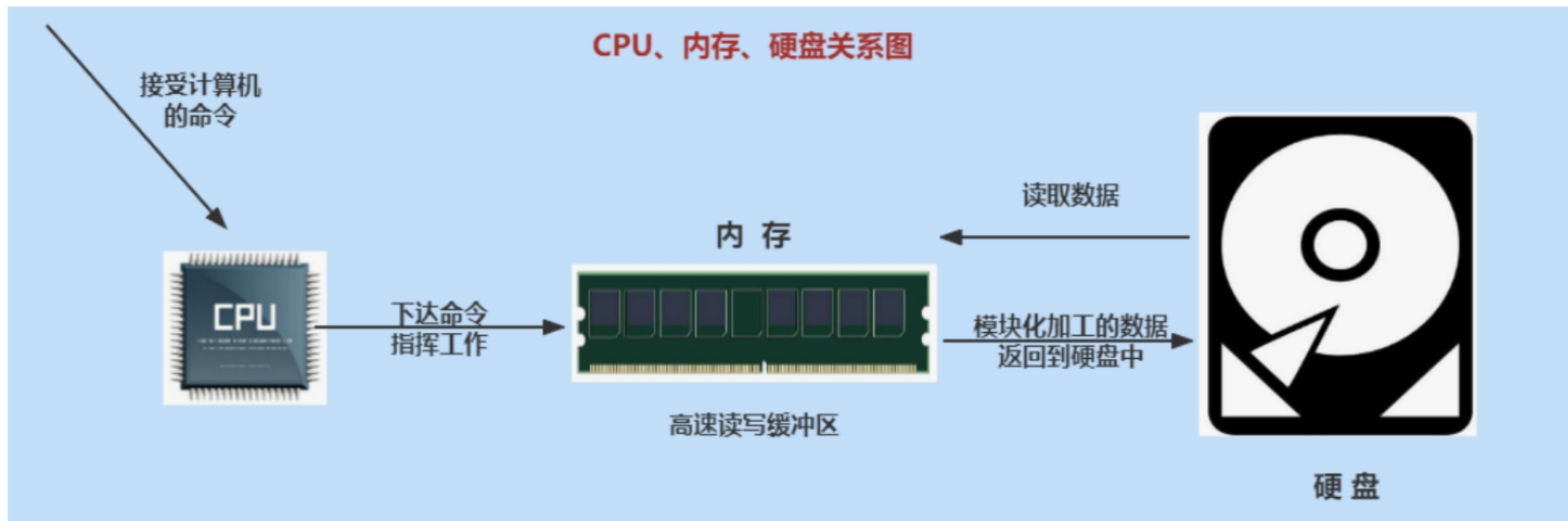


CSP-J考点详解



I/O操作：读写操作，Input/Output（输入/输出操作）是计算机系统中CPU与外部设备进行信息交换的关键过程。

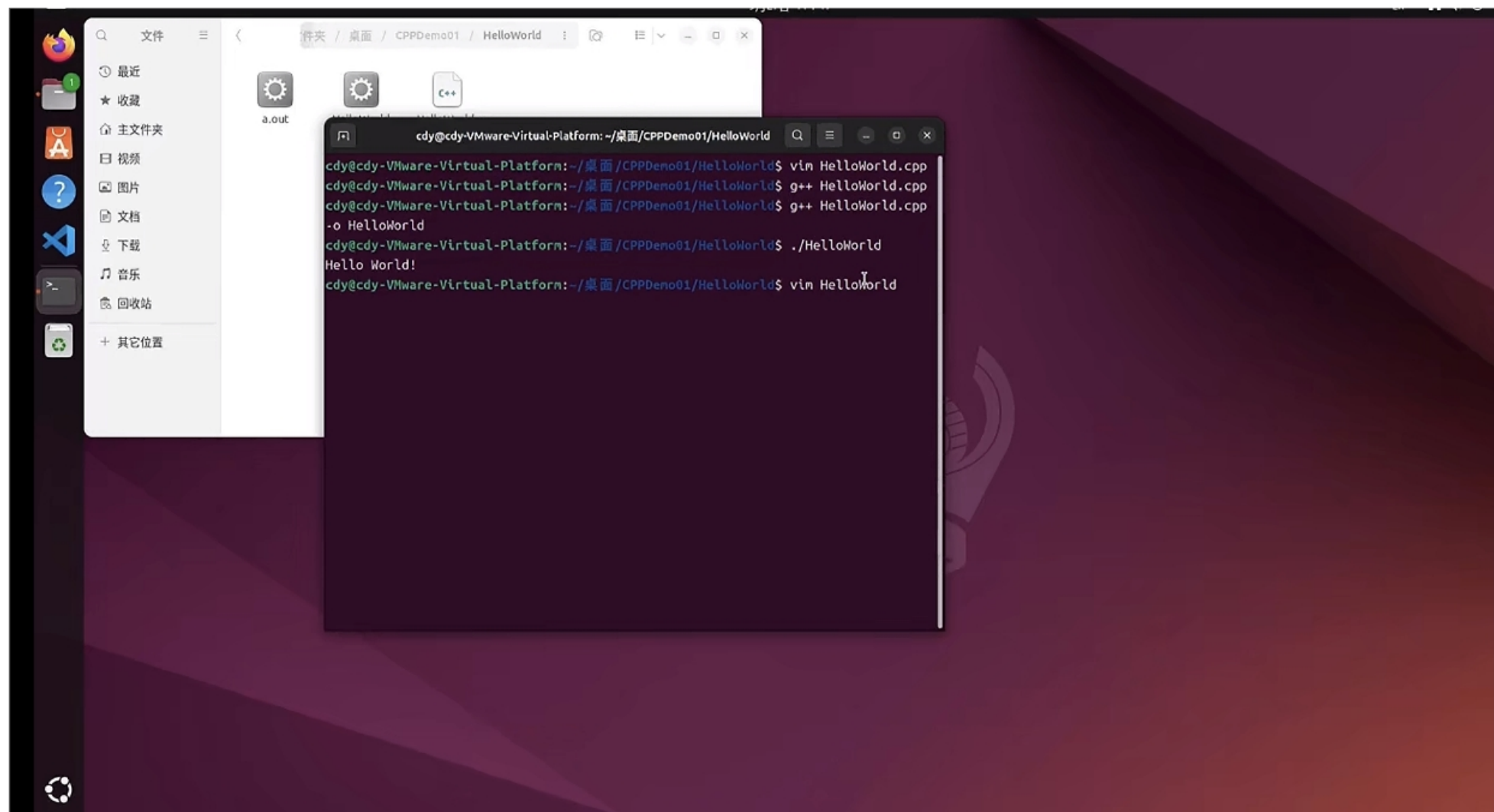
I/O设备：是计算机系统中实现数据交换的硬件装置，涵盖数据输入、输出及存储功能由操作系统统一管理。其核心分类包括字符设备（如键盘、鼠标）、块设备（如硬盘、光盘）和网络通信设备（如网卡）。
显示器？



Linux 常见操作

g++

Code::Blocks



编译与解释

- **编译型语言**

- 源代码通过**编译器 (Compiler)** 一次性转换为**目标平台的机器码 (Machine Code)**，之后可直接在目标机器上运行，无需依赖编译器或解释器。
- **核心流程**：源代码 (.c/.cpp) → 编译器 → 预处理 → 编译 → 汇编 → 链接 → 可执行文件 (.exe/.out) (c/c++)

- **解释性语言**

- 源代码通过**解释器 (Interpreter)** 逐行读取、逐行转换为机器码并执行，无需生成独立的可执行文件，执行时必须依赖解释器环境。
- **核心流程**：源代码 (.py/.js) → 解释器 → 词法分析 → 语法分析 → 语义分析 → 生成中间代码 (可选) → 逐行执行 (Java/Python)

三目运算

条件表达式 ? 表达式1 : 表达式2;

```
1 #include<bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     int a = 10, b = 20;
6     int max = (a > b) ? a : b;
7     cout << "Max is:" << max << endl;
8     return 0;
9 }
```

基本运算

- 1.算术运算符：

+： 加法运算。

-： 减法运算。

*： 乘法运算。

/： 除法运算。

%： 取模运算（求余数）。

++： 自增运算符。

--： 自减运算符。
- 2.关系运算符：

==： 等于。

!=： 不等于。

<： 小于。

>： 大于。

<=： 小于等于。

>=： 大于等于。
- 3.逻辑运算符：

&&： 逻辑与。

||： 逻辑或。

!： 逻辑非。逻辑取反
- 4.个别运算符：

=： 赋值运算符。

**： 幂运算

a	b	a && b	a b
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

位运算

符号	描述	运算规则
&	与	两个位都为1时，结果才为1
	或	两个位都为0时，结果才为0
^	异或	两个位相同为0，相异为1
~	取反	0变1，1变0
<<	左移	各二进位全部左移若干位，高位丢弃，低位补0
>>	右移	各二进位全部右移若干位，高位补0或符号位补齐

数学库常用函数

绝对值函数: `abs()`整型 `labs()`长整型

`fabs()`单精度 `fabsf()`双精度

四舍五入函数: `round(X)`

向下取整函数: `floor(X)`

向上取整函数: `ceil(X)`

平方根函数: `sqrt(X)`

```
1  #include<iostream>
2  #include<cmath>
3  using namespace std;
4
5  double roundToDecimal(double num,int decimal){
6      double factor = pow(10,decimal);
7      return round(num * factor) / factor;
8  }
9  int main(){
10     double num = 3.1415926;
11     cout << roundToDecimal(num,2) << endl;
12     cout << roundToDecimal(num,3) << endl;
13     return 0;
14 }
```

round()保留小数位做法

函数	功能描述	示例
round	四舍五入到最接近的整数	round(2.6)→3
floor	向下取整（向负无穷方向）	floor(2.6)→2
ceil	向上取整（向正无穷方向）	ceil(2.3)→3
trunc	向零方向取整（直接截断）	trunc(-2.6)→-2

三角函数：

- **功能：** 分别计算给定角度（以弧度为单位）的正弦、余弦和正切值。

- **原型：**

- `double sin(double x);`
- `double cos(double x);`
- `double tan(double x);`

- **功能：** 分别计算给定值的反正弦、反余弦和反正切值，返回值为弧度。

- **原型：**

- `double asin(double x);`
- `double acos(double x);`
- `double atan(double x);`

指数函数:

`pow(base,power)`

`a = pow(2,3);`

返回值为浮点类型，即便结果是整数

e^x

- 功能: 计算 e 的 x 次幂, 其中 e 是自然常数 (约为 2.71828)。
- 原型: `double exp(double x);`
- 示例代码:

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 int main(){
4     cout<<pow(2,3)<<endl;
5     cout<<typeid(pow(2,3)).name();
6     return 0;
7 }
```

8

d

Process exited after 0.1342 seconds with return value 0
请按任意键继续. . . |

快速幂

对数函数：

对数可以看作是指数运算的逆运算。

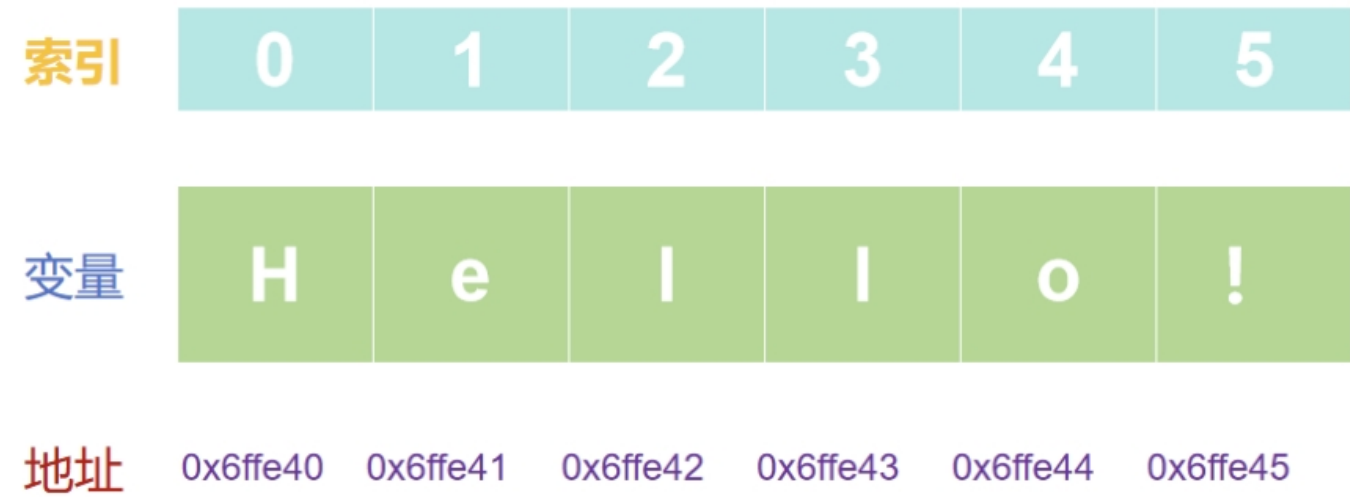
$$\log_b x = y \Leftrightarrow b^y = x$$

- 功能：log() 计算自然对数（以 e 为底），log10() 计算以 10 为底的对数。
- 原型：
 - double log(double x);
 - double log10(double x);

```
cout<<log10(10)<<endl;
```

字符数组与相关函数

```
char str1[] = "Hello!";
```



相关函数	解释
strcat(str1,str2)	把字符串2中的字符全部复制到字符数组1后
strcmp(str1,str2)	判断连个字符串是否相等（返回1， -1， 0）
strlen(str)	返回字符数组的长度（字符个数）
strlwr	大写字母变为小写字母
strupr	小写字母变为大写字母

神秘的'\0':当字符数组的空间无法满足'\0'的存在时, 会发生问题!

```
char arr1[5]={'1','2','3','4','5'};  
cout<<arr1<<endl;  
cout<<strlen(arr1)<<endl;
```

```
12345  
5
```

偶然性

```
12345???5  
11
```

cout << arr1: 当 cout 输出一个 char* 类型时, 它会从该地址开始, 一直输出字符, 直到遇到 '\0' 为止。由于 arr1 中没有 '\0', 程序会继续访问数组之后的内存, 直到碰巧遇到一个值为 0 的字节。这可能会导致输出乱码, 甚至程序崩溃。strlen 同理, 其只输出有效字符, 不包括 '\0', 但也是一直统计到 '\0' 为止。

如果需要输出实际大小, 可以用 sizeof ()

```
char arr1[5] = {'1','2','3','4','5'};  
char arr2[] = "12345";  
cout<<sizeof(arr1)<<endl;  
cout<<sizeof(arr2)<<endl;
```

```
5  
6
```

ASCII码

高四位 低四位		ASCII非打印控制字符										ASCII 打印字符													
		0000					0001					0010		0011		0100		0101		0110		0111			
		0					1					2		3		4		5		6		7			
		+进制	字符	ctrl	代码	字符解释	+进制	字符	ctrl	代码	字符解释	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符	ctrl	
0000	0	0	BLANK NULL	^@	NUL	空	16	▶	^P	DLE	数据链路转意	32		48	0	64	@	80	P	96	`	112	p		
0001	1	1	☺	^A	SOH	头标开始	17	◀	^Q	DC1	设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q		
0010	2	2	☺	^B	STX	正文开始	18	↕	^R	DC2	设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r		
0011	3	3	♥	^C	ETX	正文结束	19	!!	^S	DC3	设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s		
0100	4	4	◆	^D	EOT	传输结束	20	¶	^T	DC4	设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t		
0101	5	5	♣	^E	ENQ	查询	21	♫	^U	NAK	反确认	37	%	53	5	69	E	85	U	101	e	117	u		
0110	6	6	♠	^F	ACK	确认	22	■	^V	SYN	同步空闲	38	&	54	6	70	F	86	V	102	f	118	v		
0111	7	7	●	^G	BEL	震铃	23	↕	^W	ETB	传输块结束	39	'	55	7	71	G	87	w	103	g	119	w		
1000	8	8	◼	^H	BS	退格	24	↑	^X	CAN	取消	40	(	56	8	72	H	88	X	104	h	120	x		
1001	9	9	○	^I	TAB	水平制表符	25	↓	^Y	EM	媒体结束	41	)	57	9	73	I	89	Y	105	i	121	y		
1010	A	10	◻	^J	LF	换行/新行	26	→	^Z	SUB	替换	42	*	58	:	74	J	90	Z	106	j	122	z		
1011	B	11	♂	^K	VT	竖直制表符	27	←	^[	ESC	转意	43	+	59	;	75	K	91	[	107	k	123	{		
1100	C	12	♀	^L	FF	换页/新页	28	└	^\	FS	文件分隔符	44	,	60	<	76	L	92	\	108	l	124			
1101	D	13	♪	^M	CR	回车	29	↔	^]	GS	组分隔符	45	-	61	=	77	M	93	]	109	m	125	}		
1110	E	14	🎵	^N	SO	移出	30	▲	^6	RS	记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~		
1111	F	15	☼	^O	SI	移入	31	▼	^-	US	单元分隔符	47	/	63	?	79	O	95	_	111	o	127	␣	^Back space	

string类与相关函数

在C++中，字符串是由字符组成列。<string>头文件提供了std::string类，它是对C风格字符串的封装，提供了更安全、更易用的字符串操作功能。

可以直接 “+” 运算和 “=” 赋值

size()	返回字符串的长度（字符数）。	std::cout << str.size();
length()	与 size() 相同，返回字符串的长度。	std::cout << str.length();
empty()	判断字符串是否为空。	std::cout << (str.empty() ? "Yes" : "No");
operator[]	访问字符串中指定位置的字符。	std::cout << str[0];
at()	访问字符串中指定位置的字符（带边界检查）。	std::cout << str.at(0);
substr()	返回从指定位置开始的子字符串。	std::string sub = str.substr(0, 5);
find()	查找子字符串在字符串中的位置。	std::cout << str.find("sub") << std::endl;
rfind()	从字符串末尾开始查找子字符串的位置。	std::cout << str.rfind("sub") << std::endl;
replace()	替换字符串中的部分内容。	str.replace(pos, length, "new_substring");
append()	在字符串末尾添加内容。	str.append(" more");
insert()	在指定位置插入内容。	str.insert(pos, "inserted");
erase()	删除指定位置的字符或子字符串。	str.erase(pos, length);
clear()	清空字符串。	str.clear();
c_str()	返回 C 风格的字符串（以 null 结尾）。	const char* cstr = str.c_str();
data()	返回指向字符数据的指针（C++11 及之后的版本）。	const char* data = str.data();
compare()	比较两个字符串。	int result = str.compare("other");
find_first_of()	查找第一个匹配任意字符的位置。	size_t pos = str.find_first_of("aeiou");
find_last_of()	查找最后一个匹配任意字符的位置。	size_t pos = str.find_last_of("aeiou");
find_first_not_of()	查找第一个不匹配任意字符的位置。	size_t pos = str.find_first_not_of("aeiou");
find_last_not_of()	查找最后一个不匹配任意字符的位置。	size_t pos = str.find_last_not_of("aeiou");

函数

返回值类型 函数名（参数列表）

{

函数体（要做的事情）

return 返回结果（把结果交上去）

}

// 定义一个加法函数

int add(**int** a,**int** b)

{

int sum = a + b;

return sum;

}

形参与实参

- **形参 (形式参数)**: 在定义函数时, 写在括号里的那些变量名。它们是函数内部用来接收数据的“占位符”。
- **实参 (实际参数)**: 在调用函数时, 真正传递给函数的具体数据或变量。

```
int add(int a,int b)//这里的a,b就是形参
{
    int sum = a + b;
    return sum;
}
```



```
1  #include<bits/stdc++.h>
2  using namespace std;
3  //定义一个加法函数
4  int add(int a,int b)//这里的a,b就是形参
5  {
6      int sum = a + b;
7      return sum;
8  }
9
10 int main()
11 {
12     int x=5;
13     int y=6;
14     cout<<add(x,y)<<endl;//这里的x, y就是实参
15     return 0;
16 }
```

三种参数传递方式——值传递，引用传递，指针传递（后续）

引用传递

引用传递允许函数直接访问和修改调用处提供的变量，而不是其副本。在C++中，通过在参数类型后添加&符号来声明引用参数。

```
int add(int &a, int b)
{
    a = a + b;
    return a;
}
```

常量与变量

常量 (constant) : 是编程中一个非常重要、也非常简单的概念: **值不可改变的量**。

```
#define Max 1000 //宏常量
```

```
const int a = 10; //带类型的只读变量
```

变量 (variable) : 以某一标识符为名, 有自己的存储空间, 因此它可以被赋值, 同时它的值可以被读取和改变。

```
char c = 'H'; //字符变量
```

```
char str1[] = "Hello!"; //字符数组
```

```
int a = 1; //整型变量
```

```
float f = 0.1; //单精度浮点型变量
```

```
double ff = 0.01; //双精度浮点型变量
```

常量与变量的作用范围

作用域	定义位置	可访问范围	生命周期	示例
全局作用域	所有函数 / 类外部	整个程序	程序运行全程（全局 / 静态区）	int g_num = 10;
局部作用域	函数 / 循环 / 分支 / {} 代码块内	仅当前代码块内（出块即不可访问）	代码块执行期（栈区）	void f(){int l_num=20;}
函数参数作用域	函数参数列表	仅当前函数内	函数执行期（栈区）	void f(int p_num){}
命名空间作用域	命名空间内部	命名空间内，或通过命名空间::访问	同全局 / 局部	namespace N{int n_num=30;}
类作用域类	类 / 结构体内部	类的所有成员函数内（需对象 / 类名访问）	随对象 / 类（堆 / 栈 / 全局区）	class A{int c_num;;}

可见性：全局变量，局部变量，块变量重名时（块变量>局部变量>全局变量）

“作用域隐藏规则”

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 // 1. 全局变量（最外层作用域）
4 int num = 10;
5 void test() {
6     // 2. 局部变量（函数级作用域，覆盖全局）
7     int num = 20;
8     cout << num << endl;           // 输出20（局部覆盖全局）
9     cout << ::num << endl;         // 输出10（显式指定全局）
10    // 3. 块变量（代码块级作用域，覆盖局部）
11    if (true) {
12        int num = 30; // 块变量
13        cout << num << endl;       // 输出30（块覆盖局部）
14        cout << ::num << endl;     // 输出10
15    }
16 }
17 int main() {
18     test();
19     return 0;
20 }
```


C++中内存的5个区域

内存区域	分配/释放方式	存储内容	生命周期	特点
栈 (Stack)	编译器自动分配 / 释放	局部变量、函数参数、返回值	代码块 / 函数执行期	速度极快、空间小 (几 MB)、自动回收
堆 (Heap)	malloc/free (C 风格)	手动动态分配的任意数据	手动释放前	速度慢、空间大 (GB 级)、易内存泄漏
自由存储区	new/delete (C++ 风格)	手动动态分配的对象 / 数据	手动释放前	质是堆的一部分, 面向对象设计
全局 / 静态存储区	编译器自动分配 / 程序结束释放	全局变量、static 变量 (全局 / 局部)	程序运行全程	初始化的在.data 段, 未初始化在.bss 段
常量存储区 (.rodata)	编译器分配 / 程序结束释放	const/constexpr常量、字符串常量	程序运行全程	只读、不可修改、修改会触发崩溃

局部 const: 默认存储在栈区, const只是 “只读”, 不是 “编译期常量” (constexpr)

特别注意

```
char* str = "Hello";  
str[0] = 'X';  
cout<<str<<endl;
```

改变常量，没有输出

```
extern int number; // 声明了一个外部变量number
```

```
int main() {  
    // 1. 定义目标地址，用uintptr_t存储（适配不同系统的地址宽度）  
    uintptr_t target_addr = 0x6ffe40;  
    // 2. 将地址转为int*类型指针（reinterpret_cast用于底层类型转换）  
    int* p = reinterpret_cast<int*>(target_addr);  
    // 3. 访问/修改该地址的变量值（地址无效会导致程序崩溃）  
    cout << "地址0x6ffe40对应的值: " << *p << endl; // 输出100（若地址有效）  
    return 0;  
}
```

拓展：
输出对应地址的值

递归

递归函数是指在函数体内直接或间接调用自身的函数，本质是将一个复杂的大问题，拆解为与原问题逻辑相同但规模更小的子问题，直到子问题简化为“终止条件”，再逐层返回结果，最终解决原问题。

// 递归求阶乘

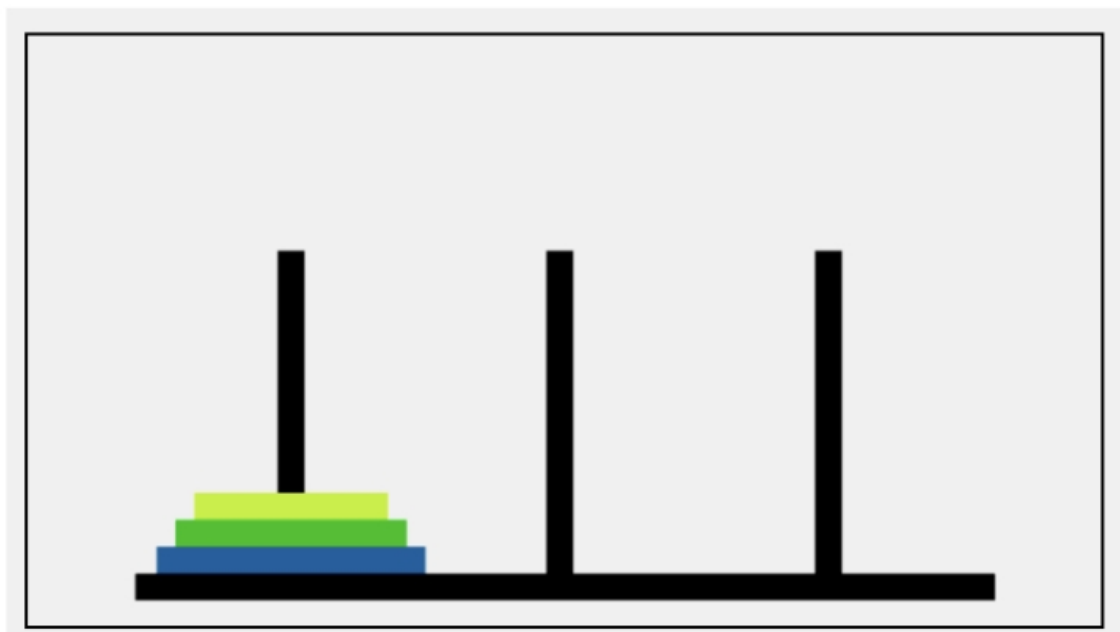
```
int factorial(int n) {  
    // 1. 终止条件: 0! 和 1! 都等于 1  
    if (n == 0 || n == 1) {  
        return 1;  
    }  
    // 2. 递归递推:  $n! = n \times (n-1)!$   
    return n * factorial(n - 1);  
}  
  
int main() {  
    cout << "5! = " << factorial(5) << endl;  
    return 0;  
}
```

递

归

递归实例——汉诺塔

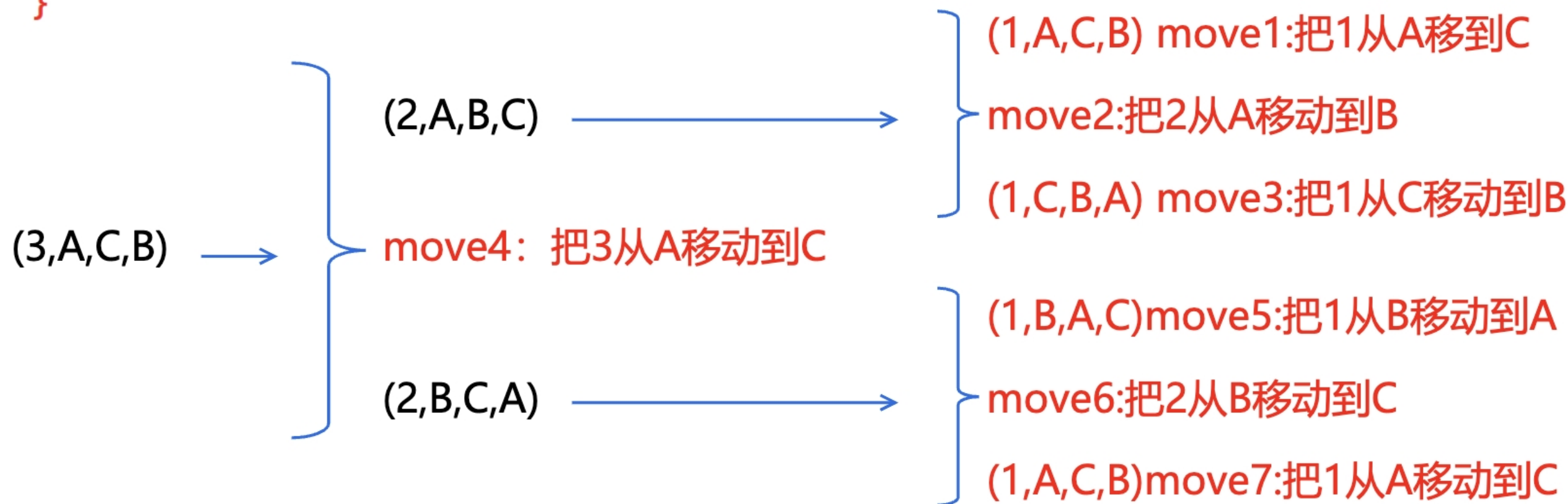
- **问题描述**：有 3 根柱子（A、B、C）和 n 个大小不同的圆盘，初始时所有圆盘按从大到小的顺序套在 A 柱上。要求将所有圆盘移到 C 柱，过程中：每次只能移动 1 个圆盘；圆盘只能从一根柱子移到另一根；任何时候，任意柱子上的圆盘都必须保持“大的在下、小的在上”。
- **核心思路**：递归 —— 将复杂问题拆解为更小的同类型问题：把 $n-1$ 个圆盘从 A 柱移到 B 柱（以 C 柱为辅助）；把第 n 个（最大的）圆盘从 A 柱移到 C 柱；把 $n-1$ 个圆盘从 B 柱移到 C 柱（以 A 柱为辅助）。



```

void hanoi(int n, char from, char to, char aux, int& step) {
    // 递归终止条件: 只有1个圆盘时, 直接从from移到to
    if (n == 1) {
        cout << "步骤 " << step++ << ": 将圆盘 1 从 " << from << " 移到 " << to << endl;
        return;
    }
    // 第一步: 把n-1个圆盘从from移到aux (以to为辅助)
    hanoi(n - 1, from, aux, to, step);
    // 第二步: 把第n个圆盘从from移到to (最大的圆盘, 直接移)
    cout << "步骤 " << step++ << ": 将圆盘 " << n << " 从 " << from << " 移到 " << to << endl;
    // 第三步: 把n-1个圆盘从aux移到to (以from为辅助)
    hanoi(n - 1, aux, to, from, step);
}

```



结构体

//定义结构体

```
struct student
{
    string name;
    int age;
    int score;
};
```

//创建结构体数组

```
struct student stuarr[3] = {
    {"张三",18,100},
    {"李四",17,90},
    {"王二",16,80}
};
```

//下标访问结构体数组中的元素

```
stuarr[2].name = "李三";
stuarr[2].age = 17;
stuarr[2].score = 10;
```

联合体

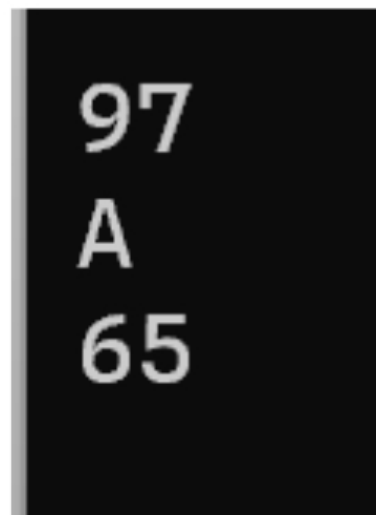
结构体(struct)中所有**变量**是“共存”的——优点是“有容乃大”，全面；缺点是struct内存空间的分配是粗放的，不管用不用，全分配。

联合体(union)中**各变量**是“互斥”的——缺点就是不够“包容”，即任何两个成员不会同时有效；但优点是内存使用更为精细灵活，也节省了内存空间。

```
union Mark{ //表示成绩的联合体，同一门课程只会存在一种成绩表示方法
    char level; // 表示等级制的成绩‘ A’ ‘ B’ ‘C’ 等
    bool pass; // 只计是否通过课程的成绩 0 1
    int grade; // 表示分数制的成绩
}cj[3];
```

联合体中数据是共享相同的存储空间，各种变量名都可以同时使用，操作也是共同生效

```
int main(){  
    cj[0].grade = 97;  
    cout<< cj[0].grade<<endl;  
    cj[0].level = 'A';  
    cout<< cj[0].level<<endl;  
    cout<< cj[0].grade<<endl;  
    return 0;  
}
```



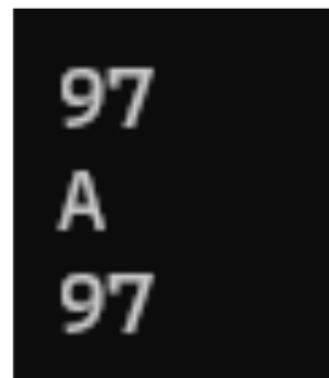
97
A
65

grade: 4字节内存存储的是int类型97的二进制: 0x00 0x00 0x00 0x61

level: 字符型, 覆盖了4字节内存的第1字节, 内存变为: 0x00 0x00 0x00 0x41

如果将grade的类型从int变为double，结果会发生改变，为什么？

```
union Mark{  
    char level;  
    bool pass;  
    double grade;  
}cj[3];
```



97
A
97

grade: 8字节内存存储的是double类型97的二进制: 0x40 0x58 0x40 0x00 0x00 0x00 0x00 0x00

当你给level (char, 1 字节) 赋值 'A' (ASCII 65=0x41) 时, 仅覆盖了 8 字节中的第一个字节

此时 8 字节变为: 0x40 0x058 0x40 0x00 0x00 0x00 0x00 0x41, 对应的double值不再是 97, 而是一个接近 97 但有微小偏差的数。

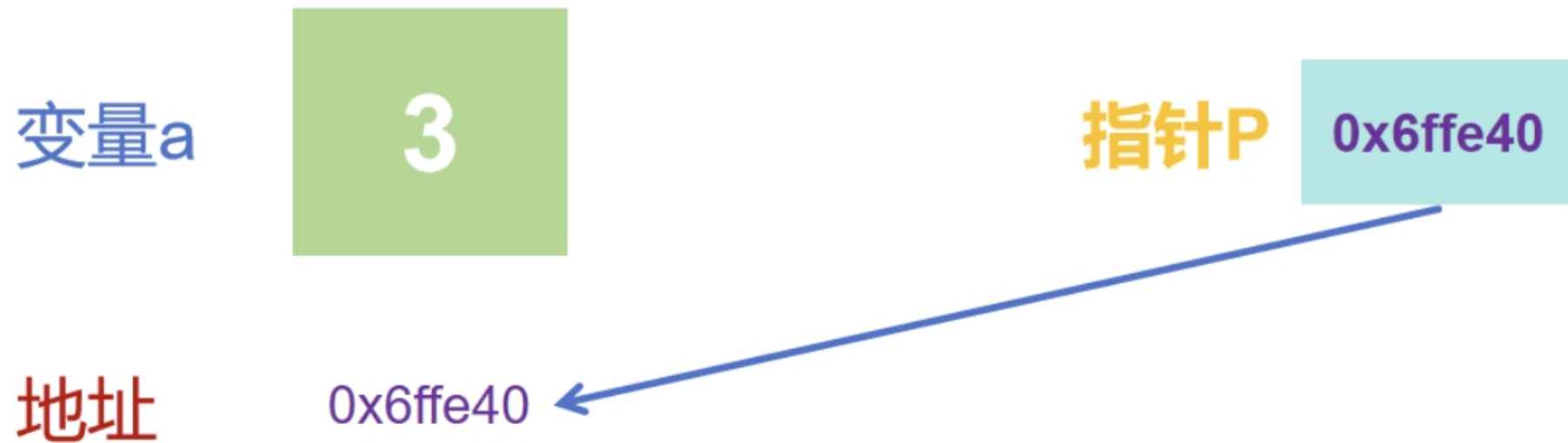
double 类型占用**8 字节（64 位）**，这 64 位被划分为 3 部分

部分	位数	作用
符号位 (S)	1 位	0 表示正数, 1 表示负数 (97 是正数, 所以 S=0)
指数位 (E)	11 位	存储偏移指数 (偏移量 = 1023, 即实际指数 = 存储指数 - 1023)
尾数位 (M)	52 位	存储有效数字 (尾数), 默认整数部分是 1, 只存储小数部分 (隐含 1 机制)

存储公式: $value = (-1)^S \times (1 + M) \times 2^{(E-1023)}$

指针

指针是C/C++语言中最强大也最具挑战性的特性之一。它可以表示各种数据结构，像汇编语言一样处理内存地址。（在64位操作系统中指针占8字节空间）



- **获取地址：** &变量名
- **解引用：** *指针变量，访问指针指向的内容

```
#include<bits/stdc++.h>
using namespace std;
int main()
{
    int a = 3;
    int *p = &a;
    cout<<a<<endl;
    cout<<*p<<endl;
    cout<<p<<endl;
    return 0;
}
```

```
3
3
0x6ffe54
```

基于指针的数组访问

在 C++ 中，数组名本质上是一个指向数组第一个元素的常量指针（不能被修改指向）。因此，我们可以通过指针的算术运算来访问数组的任意元素，这和用下标 [] 访问是等价的。

数组名 等价于 数组的首地址

```
int arr[5] = {10, 20, 30, 40, 50};  
// arr 等价于 &arr[0], 指向数组第一个元素  
cout << "通过数组名+偏移访问: " << endl;  
for (int i = 0; i < 5; i++) {  
    // *(arr + i) 等价于 arr[i]  
    cout << "arr[" << i << "] = " << *(arr + i) << endl;  
}
```

通过数组名+偏移访问：

```
arr[0] = 10  
arr[1] = 20  
arr[2] = 30  
arr[3] = 40  
arr[4] = 50
```

```
cout << "通过自定义指针遍历：" << endl;
int* p = arr; // 指针p指向数组第一个元素（等价于 p = &arr[0]）
for (int i = 0; i < 5; i++) {
    cout << "当前元素：" << *p << endl;
    p++; // 指针自增，指向数组的下一个元素（偏移量为数组元素的类型大小）
}
```

```
通过自定义指针遍历：
当前元素： 10
当前元素： 20
当前元素： 30
当前元素： 40
当前元素： 50
```

```
// 指针也可以用[]下标访问，p[i] 等价于 *(p + i)
p = arr; // 重置指针指向数组开头
cout << "指针下标法访问：" << endl;
for (int i = 0; i < 5; i++) {
    cout << "p[" << i << "] = " << p[i] << endl;
}
```

```
指针下标法访问：
p[0] = 10
p[1] = 20
p[2] = 30
p[3] = 40
p[4] = 50
```


字符指针

字符指针 `char*` 是指向**单个字符**的指针，但它最常用的场景是指向以 `'\0'` **结尾的字符数组**的首地址。这是它和普通指针最大的区别：普通指针关注单个元素，而字符指针通常关联一个 “字符序列 + 结束符” 的整体。

```
const char* str1 = "hello world"; // 推荐加 const, 避免误修改
cout << "str1 = " << str1 << endl; // 直接输出整个字符串 (直到'\0')
cout << "str1的首字符: " << *str1 << endl; // 解引用取第一个字符 'h'
cout << "str1的第二个字符: " << *(str1 + 1) << endl; // 'e'
cout << "str1的长度: " << strlen(str1) << endl; // 11 (不含'\0')
```

```
str1 = hello world
str1的首字符: h
str1的第二个字符: e
str1的长度: 11
```



```
char arr[] = "c++ pointer"; // 字符数组，存储在栈上，可修改
char* str2 = arr; // 指针指向数组首地址
str2[0] = 'C'; // 合法：修改第一个字符
cout << "str2 = " << str2 << endl; // 输出 "C++ pointer"
// 遍历字符串（通过指针）
cout << "遍历str2: ";
while (*str2 != '\0') { // 直到遇到结束符'\0' 停止
    cout << *str2;
    str2++; // 指针后移
}
cout << endl;
```

```
str2 = C++ pointer
遍历str2: C++ pointer
```

// 3. 动态分配字符数组 (堆内存)

```
char* str3 = new char[20]; // 分配20个字符的空间  
strcpy(str3, "dynamic string"); // 复制字符串到堆内存  
cout << "str3 = " << str3 << endl;  
delete(str3); // 必须释放堆内存, 避免内存泄漏  
str3 = nullptr; // 置空, 防止野指针
```

```
str3 = dynamic string
```

空指针和野指针

- **空指针 (nullptr)** : 明确指向 “无有效地址” (地址0) , 解引用会崩溃, 但比野指针安全 (可判断 if (str == nullptr)) 。
- **野指针**: 未初始化 (char* p;) 或释放后未置空 (delete(p); 后 p 仍指向原地址) , 访问会导致不可预测的错误。

// 4. 空字符指针与野指针

char* str4 = NULL; // 空指针 (指向地址0, 无有效内存)

//cout << *str4; // 错误! 解引用空指针会崩溃

char* str5; // 未初始化的野指针 (指向随机地址) , 绝对禁止直接使用

- 普通指针（如 `int* p`）输出 `cout << p` 会打印地址，但遇到 `char*` 时，`cout` 会自动从指针指向的地址开始，逐个打印字符，直到遇到 `'\0'`（空字符）为止。
- 如果想打印字符指针的地址，需要强制转换：`cout << (void*)str1 << endl;`。

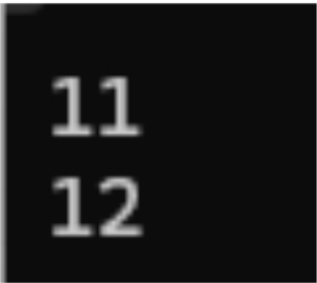
```
int arr1[5]={1,2,3,4,5};  
cout<<str1<<endl;  
cout<<arr1<<endl;  
cout<<(void*)str1<<endl;
```

```
hello world  
0x6ffe10  
0x488001
```

注意：同理，字符数组也可以直接通过数组名（`char*`）输出

strlen与sizeof：字符的长度和实际占用空间的长度

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    const char* str = "hello world";
    cout<<strlen(str)<<endl;
    cout<<sizeof("hello world") <<endl;
    return 0;
}
```



11
12

sizeof：所占用的空间（字节数）

指向结构体的指针

```
#include <iostream>
#include <cstring>
using namespace std;
struct Student {
    char name[50];
    int age;
    float grade;
};
int main() {
    Student s1[2] = {{ "Alice", 20, 80.2 }, { "Ben", 25, 90.1 } };
    // 声明指向结构体的指针
    Student *ptr;
    // 让指针指向结构体变量
    ptr = s1;
    // 通过指针访问结构体成员
    // 方法1: 使用箭头运算符 ->
    cout << "Name: " << ptr->name << endl;
    cout << "Age: " << ptr->age << endl;
    cout << "grade: " << ptr->grade << endl;
    // 方法2: 解引用后使用点运算符 .
    cout << "Name: " << (*ptr).name << endl;
    return 0;
}
```

```
Name: Alice
Age: 20
grade: 80.2
Name: Alice
```

引用

引用可以理解为**变量的“别名”** —— 给一个已经存在的变量起一个新名字，操作这个别名就等同于操作原变量本身（二者共享同一块内存空间）。

数据类型 &引用名 = 原变量名;

- 1.& 在这里不是“取地址符”，而是**引用符号**；
- 2.引用必须**在定义时立即初始化**（不能先定义再赋值）；
- 3.引用一旦绑定原变量，就不能再绑定其他变量（终身绑定）

引用作为函数参数（避免拷贝，高效传参）

如果函数参数是“值传递”，会拷贝一份新数据（耗时耗内存）；用引用传参，直接操作原变量，效率更高

```
void swap(int &x, int &y) {  
    int temp = x;  
    x = y;  
    y = temp;  
}  
  
int main() {  
    int m = 5, n = 10;  
    swap(m, n);  
    cout << "m=" << m << ", n=" << n << endl;  
    return 0;  
}
```

m=10, n=5

引用VS指针

特性	引用 (Reference)	指针 (Pointer)
初始化	必须初始化	可先定义后赋值
指向变更	不能更换绑定的变量	可指向不同变量
空值	不能为 NULL	可以为 NULL (空指针)
语法	用 & 定义, 使用时无额外符号	用 * 定义, 使用时需解引用 *p
内存占用	不占用额外内存 (语法糖)	占用内存 (64 位系统 8 字节)

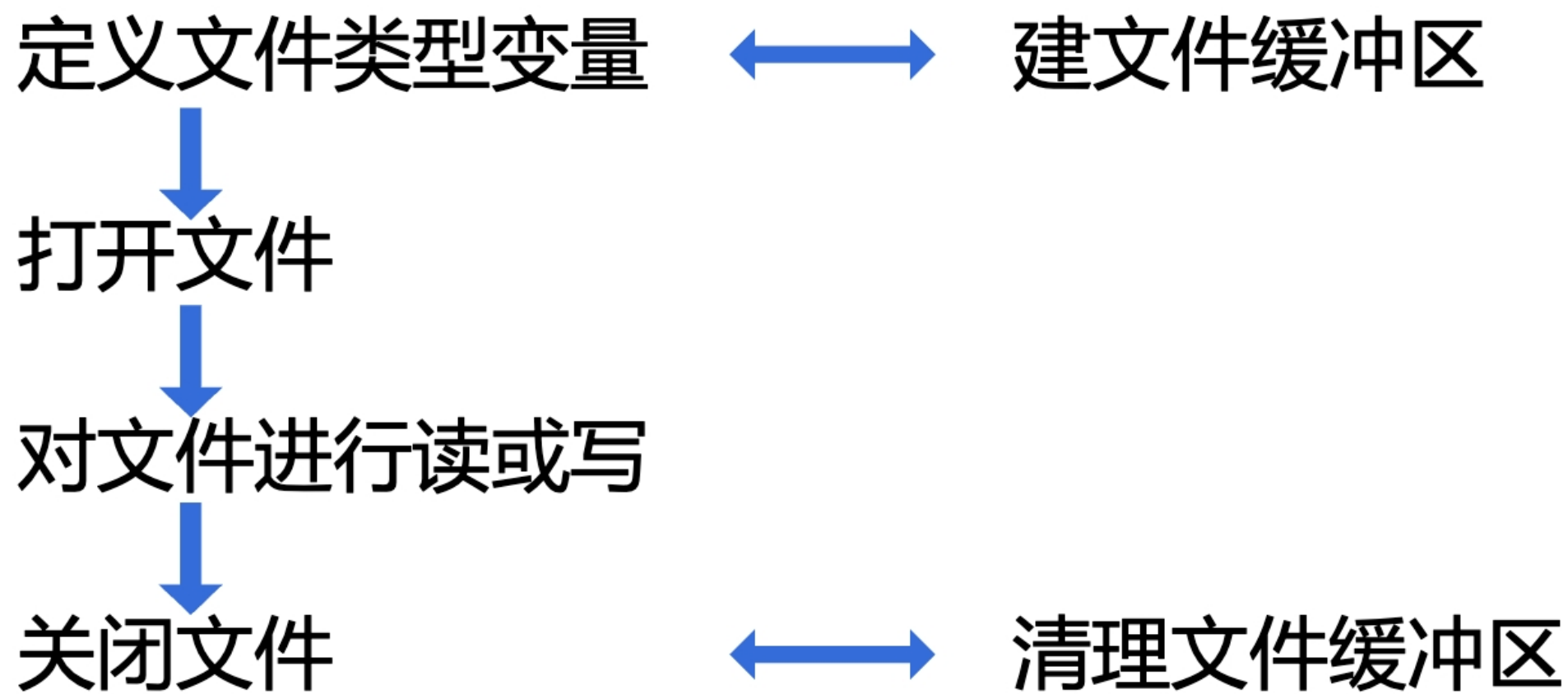
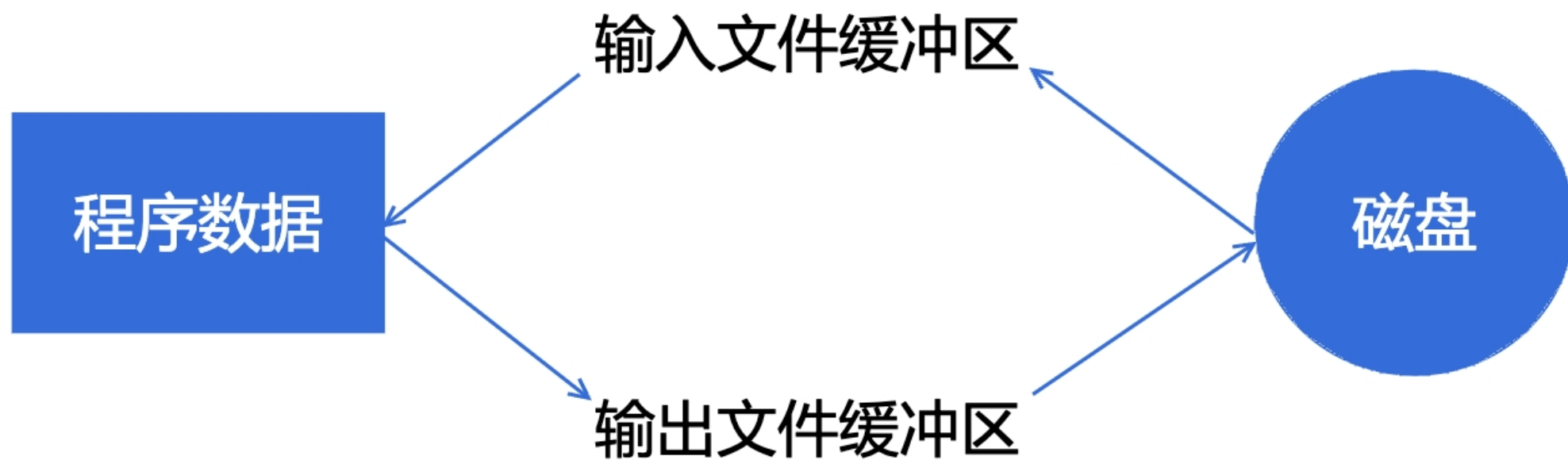
底层逻辑：引用在编译器编译时，会被转换成一个**不能修改指向的指针**（只读指针）

文件及基本读写

文件：存储在外部介质上数据的集合，简单来说就是把数据通过字节序列保存在磁盘上。

文本文件：内容是文字（哪国语言都行），用记事本打开能看到文字的文件。

二进制文件：本质上所有文件都是0,1串，因此都是二进制文件。但是一般内容不是文字，记事本打开看是乱码的文件，称为二进制文件。word文件记事本打开看是乱码，因此也是二进制文件。



FILE * fopen(char * filename, char * mode)

以mode模式打开文件 filename , 如果打开失败, 返回NULL
如果成功, 返回一个 FILE * 指针, 以后对文件的操作均通过该指针进行。

FILE 是一个struct, 记录被打开文件的各种信息。

打开失败的原因: 文件不存在, 文件名不合法, 试图以写方式打开只读文件, 没有权限.....

fopen函数mode参数取值:

"r": 读模式。打开文件用于读取信息。

"w": 写模式。创建打开文件用于写入信息。如果filename已经存在, 则原文件立即丢失。

"a": 添加模式。打开已有文件, 在尾部添加新内容。如果文件不存在, 则创建之。所有写入操作的效果都是往尾部添加内容。

"r+": 读写模式。打开文件, 可读可写。如果文件本来存在, 则原内容不会丢失; 如果文件本来不存在, 打开失败。

打开模式中加"b"即指定为二进制方式打开, 如 "rb","wb","ab","r+b"
否则称为以文本方式打开。

文件读写操作

```
#include <cstdio>
#include <iostream>
using namespace std;
int main(){
    FILE * fpin=fopen("in.txt","r");//定义文件指针
    FILE * fpout=fopen("out.txt","w");
    int n;
    int m;
    fscanf(fpin,"%d%d",&n,&m);//从文件中获取输入
    cout<<n<<" "<<m;
    fprintf(fpout,"%d",m+n);//输出和到另一个文件
    return 0;
}
```